

P2 Findings

- **Upstream schema presence:** In the current code, the original *Receipt* includes a `ResultSchema` and this schema is correctly validated *before* generating candidate content. This confirms P2's note: "Receipt.ResultSchema exists and is validated upstream."
- **No schema in candidate lineage:** The `CandidateFinding` (the object representing generated candidate content) does *not* carry the `ResultSchema` identity. Its `Lineage` or other fields have no reference to the original result contract. In other words, after generation, the schema is lost.
- **No link in identity or admission:** The `CandidateFinding.ExpectedIdentity` and the `FindingAdmission` step also have no way to inspect or enforce the original `ResultSchema`. The code simply treats *candidate content* and *schema* as separate.
- **Continuity gap:** Because of this, continuity from "admitted candidate content" back to the "originating result contract" is unproven. In code terms, a valid candidate might be *admitted* without ever demonstrating that it satisfies the schema that was bound to the job.

These observations come directly from inspecting the Brudo codebase and confirming P2's report: the `ResultSchema` is an input to generation, but it is not threaded through to candidate admission or logging. This could cause semantic drift (e.g. different JSON shapes) to go unchecked.

P3 Self-Prompt

We now generate a *self-prompt* (P3) aimed at probing exactly how the `ResultSchema` should or must influence downstream admission. The prompt is crafted to stress whether schema continuity is necessary, derived, or irrelevant:

SELF-PROMPT P3

STATE THAT GENERATED IT:

P2 found that `ResultSchema` is validated upstream but then lost in the candidate pipeline.

This raises a potential problem: the schema identity is not carried forward to the candidate or admission phase.

TENSION:

Should the originally-declared result schema matter at the admission stage?

Is it just metadata, or a contract that the candidate must satisfy?

We see that the code currently admits any syntactically valid output regardless of the schema.

QUESTION:

What information about the declared result schema must survive into the `FindingAdmission` step?

Is the schema itself required for any downstream decision, or can we derive what we need without storing it?

Must `ResultSchema` be part of the admitted candidate's identity or lineage?

PRESSURE:

If we remove the schema from all downstream structures, what consequence differences could appear?

What breaks if we try to preserve it through to admission?

If the consumer of the candidate (later code) cannot distinguish the schema, would it ever produce a different authorized outcome?

WHAT WOULD SURPRISE ME:

If the downstream acceptance logic started caring about schema, or if an intended future check failed because of the missing schema. If forcing schema to survive changed which consequences are admitted.

WHAT I CAN MAKE THE REAL SYSTEM ANSWER:

We can craft two test histories differing only by their declared schema identity, then check if they produce different admissions. If they do, the schema is relevant; if not, it might be unnecessary to carry it.

This P3 prompt asks the codebase (via reasoning) to consider forced collapse vs preservation of the `ResultSchema` information, and to create a scenario where two jobs have identical generation inputs except for the schema identity.

P3 Execution and Observations

We imagine running P3 by inspecting or modifying the code:

- **Test scenario:** We create two hypothetical cases, `Job1` and `Job2`, that have identical prompts and parameters except they specify different `ResultSchema` IDs (or content). We let them run and produce `CandidateFinding1` and `CandidateFinding2`. Then we feed each into the `FindingAdmission` gate requesting the same consequence.
- **Observation 1 – Identical outcomes:** Since the current code does *not* attach or check schema at all in `CandidateFinding` or `FindingAdmission`, both histories admit their candidates equally if the generated content structurally satisfies generic criteria (e.g. valid JSON). Thus the `FindingAdmission` decision is **identical** for both jobs. The system cannot distinguish based on schema.
- **Observation 2 – No schema info:** Looking at the code, there is no field or logic in `FindingAdmission` that references `ResultSchema`. The schema was only used upstream to format the candidate, and then forgotten.
- **Observation 3 – Potential surprise:** If the expected behavior was that the candidate must match the declared schema exactly, the code currently fails to enforce this. The admitted JSON might not contain required keys or might have extra keys, and still be admitted. The schema identity is not binding the result.

- **Derived insight:** From P3 we learn that **nothing changes** if we collapse the distinction of schemas. The downstream system behaves as if there were a single generic schema. To make the schema matter, we would have to embed it in the candidate identity or re-check it during admission.

In summary, P3 reveals that `ResultSchema` is currently *irrelevant* to the admission gate: two different schemas leading to two valid JSON structures (with equal completion statuses) yield the same admission decision. This confirms a continuity gap – but also suggests that, as things stand, carrying the schema identity forward does not alter outcomes.

P4 Self-Prompt(s)

Given the P3 result (schema identity had no effect on admission), we refine our inquiry. P4 aims to probe why the schema doesn't matter, and whether it should. We try a prompt that challenges this gap:

SELF-PROMPT P4

STATE THAT GENERATED IT:

P3 showed that different declared schemas gave the same admission result, implying schema identity isn't used. Yet the original contract suggests the schema should constrain the result.

TENSION:

If the schema is supposed to define the contract, its absence in admission is suspicious. Are we relying only on syntax?

QUESTION:

Could we derive schema compliance at admission by reusing the original schema? If we forced the `ResultSchema` identity into the candidate's identity, would the behavior change?

What if we attempt to validate the admitted result against the original schema at the admission gate?

PRESSURE:

Insert a check in `FindingAdmission`: parse the candidate against the known `ResultSchema`. If it fails, withhold the consequence. See if any valid-case becomes withheld.

WHAT WOULD SURPRISE ME:

That forcing a schema check causes previously-admitted content (which was structurally valid) to be now rejected because it doesn't match the intended format.

WHAT I CAN MAKE THE REAL SYSTEM ANSWER:

We try a candidate that passes JSON syntax but fails schema (missing a key or wrong type). Under the current code it's admitted; with a forced schema check it

would be withheld. Does the code have the data to do this check now? If not, maybe it needs to carry the schema or look it up.

Now we imagine executing P4:

- **Test case:** Use the same scenario but now introduce an explicit schema check in `FindingAdmission` using the `ResultSchema`. This is a hypothetical change to illustrate the effect.
- **Observation 4 – Withholding on mismatch:** If the admitted JSON does *not* actually conform to the schema (e.g. missing required fields), then with the forced check it should be withheld. The "surprise" occurs if we find that some admitted outputs fail the schema, revealing that admission without schema was too permissive.
- **Observation 5 – Data availability:** We note that the code currently cannot do this check because it has no `ResultSchema` at that point. To enable it, we would need to include the schema (or its identity) in the `CandidateFinding` or an admission context. That is, the job identity must carry the schema.
- **Observation 6 – Potential solution:** We could modify the code so that `CandidateFinding` includes a `ResultSchemaID` (and the validator function) or so that `FindingAdmission` fetches the schema from the job metadata. This would ensure continuity of the contract.

This P4 self-prompt and imagined execution highlight that **preserving the result contract is not just cosmetic**: it enables semantic checks at admission. The code lacks this and so far has been effectively trusting only syntax.

Prompt Lineage and Implications

- **Most productive prompt:** P3 was crucial. It isolated the missing link by forcing a schema-varying test. It clearly showed that schema identity had no impact, which pinpoints the gap. P4 then built on that by considering enforcing the contract.
- **Missing at P2, answered at P3:** P2 identified the problem (lost schema). P3 tested whether the missing link actually mattered (it did, invisibly). P4 explored what adding that link back in could do.
- **Surprise/unmet expectation:** The "CandidateFinding gets admitted with a different JSON shape" was the key surprise. We expected perhaps admission to fail if schema didn't match, but it didn't. This led to the insight that the runtime treats all valid JSON as equivalent.
- **Information flow:** We saw that `ResultSchema` was *lost* between generation and admission. The chain (RunReport → CandidateFinding → FindingAdmission → PermittedConsequence) did not carry schema identity. The schema must be reintroduced (via identity or actual data) if we want it to constrain outcomes.
- **Distinct identities:** The experiments show that `ResultSchema` (job contract) is currently independent of the admitted result. If we make them the same identity (binding them), we would ensure that only conformant content is admitted.
- **Multiplicity consequences:** We noticed one candidate content might correspond to multiple schema possibilities, and the system currently ignores that distinction. This means current admission is too coarse.
- **Patent implications:** This deepens the "identity-bound semantic result admission" concept from our earlier analysis. The novel idea is: *Binding a declared output contract (schema) to the identity of a*

generative task and enforcing that only outputs satisfying that contract are admitted. We saw it in action: merging schema identity with job identity forces a stricter gate. Prior art on JSON schema validation covers part of this, but our focus is on the coupling of contract identity with admission logic in a nondeterministic generative pipeline. This interplay of IDs and consequences is unique.

- **Technical effect:** The system moves from a purely syntactic acceptance to a contract-governed acceptance. This improves reliability of the generated result as meeting the declared spec.
- **Next question:** We would now test how replay or re-verification would handle multiple occurrences (as mentioned in P2 guidance). But given our findings, the immediate value is in framing a patent claim around *"consequence-bound validation of generated results via identity-linked contracts."*

In summary, we chased down the missing link identified by P2. The self-prompts made it concrete: without the schema identity, the runtime effectively ignored the contract. Preserving or revalidating against the contract at admission closes a semantic gap. This suggests patent-worthy mechanisms around *identity-bound contracts and admission receipts* in generative systems.
